



CNC Plotter



Automation & Drawing Pipeline

Team: Hamza Abo Alzahab, Abdalrhman Rasheed.

Coach: Tariq AlSabbagh ALTajy

Damascus University – The Faculty of Informatics Engineering -
Department of Artificial Intelligence –
ITE Robotic Club

Technical Documentation
Built on Arduino + GRBL · Powered by Python

1. Overview

This project focuses on automating the workflow of a CNC plotter. During the construction of our plotter, we faced a repetitive and time-consuming process every time we wanted to start a new drawing. To solve this, we developed a software tool that streamlines the entire procedure.

The system integrates a custom-built CoreXY plotter with an intelligent Python-based pipeline that handles everything from image ingestion to G-code transmission. The result is a seamless end-to-end drawing experience that dramatically reduces setup time and manual intervention.

With this application, you can:

- Select an image from your device
- Capture a new image using your webcam

- Choose from multiple drawing styles (Squiggle Sin, Vector/Canny Edge, TSP, Voronoi, CMYK Layers)
- Send the processed output directly to the CNC plotter then wait for your drawing.

Our goal is to simplify the drawing process, reduce manual setup time, and provide a smoother experience for users working with custom-built CNC plotters.

Contents

1. Overview	1
With this application, you can:	1
2. Features	3
3. Mechanical Design (Hardware)	3
Motion System	3
Pen Axis (Z-Axis).....	3
Pressure Suspension Mechanism.....	4
Color-Change Mechanism.....	4
Cooling & Enclosure.....	5
4. Electronics & Power System	5
Microcontroller	5
Stepper Drivers	5
Power Supply	5
5. Software Architecture	6
Firmware (GRBL)	6
Python Application Pipeline	6
Drawing Modes	7
Buffered Streaming Protocol.....	9
Controllable G-code Streaming.....	9
6. Challenges & Solutions	10
7. Results.....	11
8. Future Improvements.....	12

2. Features

Feature	Description
Image Selection	Load any image from your device through a file dialog for processing and drawing.
Webcam Capture	Capture a live photo directly from your webcam without needing external tools.
Multiple Drawing Styles	Choose between Squiggle/Sin wave, Vector/Canny Edge, TSP, Voronoi, and CMYK Layer modes.
CMYK Color Printing	Converting images into Cyan, Magenta, Yellow, and Black channels for multi-color drawing passes.
G-code Generation	Automatically converts processed image data into GRBL-compatible G-code commands.
Buffered Streaming	Sends G-code using buffered streaming for smooth, uninterrupted plotter motion.
Pygame GUI	A responsive graphical interface for real-time control and drawing style selection.
SVG Export	Converts drawing paths into SVG vector format for further editing or archiving.
TSP & Voronoi Art	Pillow-based algorithms generate Travelling Salesman Problem paths and Voronoi stippling effects.
Pressure Suspension	Custom bearing-based arm absorbs excess pen pressure to protect the paper and prevent motor stalls.

3. Mechanical Design (Hardware)

Motion System

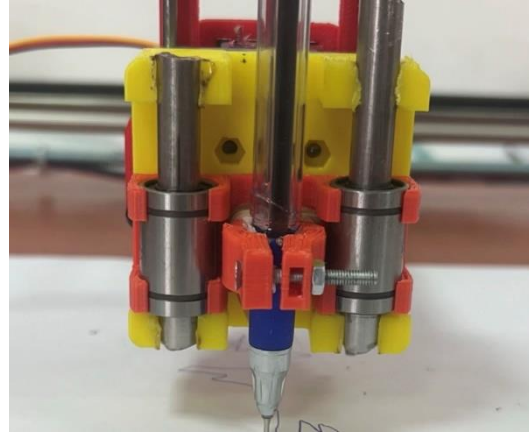
The plotter uses a CoreXY (H-Bot) kinematic structure driven by two NEMA17 stepper motors. Both motors cooperate to move the print head along X and Y simultaneously, yielding faster and more precise movement than a traditional Cartesian setup.

Pen Axis (Z-Axis)

An MG90s servo motor controls pen lift. The servo receives PWM signals from the GRBL firmware to raise the pen (pen-up) or press it onto the paper (pen-down) during drawing transitions.

Pressure Suspension Mechanism

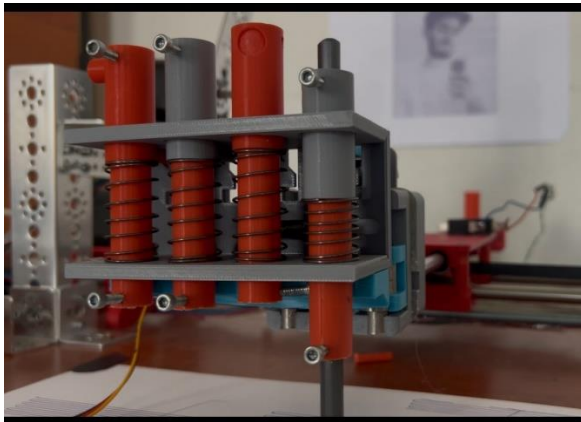
A custom mechanical assembly handles excessive pen pressure. Bearings allow the pen holder to rise automatically when downward force exceeds a threshold — absorbing shock, preventing paper damage, avoiding motor stalls, and maintaining smooth movement. The same principle was applied to the color-change mechanism.



Color-Change Mechanism

The color-change mechanism is inspired by the fan speed button assembly, where pressing one button physically releases all others. Four pens of different colors are mounted in a shared holder. When one pen is pressed down, the spring-loaded mechanism automatically pushes the remaining three pens up, ensuring only the active pen makes contact with the paper at any given time.

The pens are actuated by a fixed NEMA 17 stepper motor mounted at the corner of the CNC frame. This motor is connected to the Z-axis output on the CNC shield and drives downward to engage the selected pen. However, the current spring mechanism exerts more resistive



force than the NEMA 17 can overcome, preventing the motor from fully pressing the pens into the drawing position.

Two solutions are being evaluated to resolve the force imbalance:

- Replace the NEMA 17 with a higher-torque motor capable of overcoming the spring resistance.
- Substitute a lighter spring so the existing NEMA 17 can actuate the mechanism with its current torque output.

Cooling & Enclosure

All electronics are housed in a closed enclosure with an active cooling fan to ensure thermal stability during long sessions and prevent component failure.



4. Electronics & Power System

Microcontroller

An Arduino Uno paired with a CNC Shield v3 serves as the main controller, providing breakout headers for stepper drivers and I/O pins for the servo and limit switches.

Stepper Drivers

TMC2208 drivers replaced the initial A4988 modules. The TMC2208 operates with significantly lower heat output, near-silent stepping via StealthChop, and more stable current delivery.

Power Supply

The system was upgraded to a 24V, 5A supply. Each NEMA17 motor draws approximately 1.2A; the original 12V 3A supply caused torque loss and instability. Running at 24V improves torque and acceleration without sacrificing reliability.

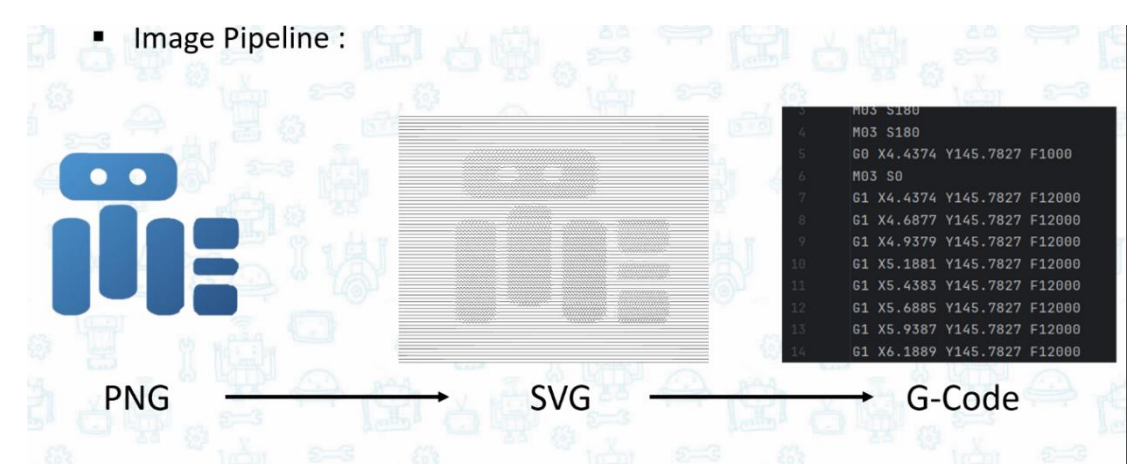
Component	Model / Spec	Role
Microcontroller	Arduino Uno + CNC Shield	Motion control & G-code interpretation
Stepper Motors	NEMA17	X and Y axis movement
Stepper Drivers	TMC2208	Quiet, efficient motor driving
Servo Motor	MG90s	Pen up/down control (Z-axis)
Power Supply	24V – 5A	Full system power

5. Software Architecture

Firmware (GRBL)

A modified version of GRBL is used as the firmware. The official release does not natively support CoreXY kinematics or servo control. After evaluating several community-patched builds, a version was adopted that supports both CoreXY motion planning and servo PWM output on the Z-axis pin.

Python Application Pipeline



The application is structured as a modular Python pipeline. The table below lists the key components and their roles:

Module / Library	Purpose
pygame	Graphical user interface to render buttons, previews, and handles all user input events.
GRBLUploader	Core transmission module — manages serial connection to Arduino and sends G-code via buffered streaming.
Gcode_Converter.py	Transforms processed image coordinates and paths into valid GRBL-compatible G-code instructions.
Svg_Converter.py	Converts drawing paths into SVG vector format for export or further editing.
svgpathtools	Parses SVG path data — extracts bezier curves and line segments, then samples them into (x, y) coordinates for G-code conversion.
Pillow (PIL)	Core image processing library used for loading, resizing, filtering, and preparing images for TSP and Voronoi style algorithms, and decomposing CMYK from images.
OpenCV	Used in Vector/Canny mode for edge detection, contour extraction, and capturing images .
serial (pyserial)	Handles low-level serial communication between the Python app and the Arduino over USB.
tkinter (filedialog)	Provides the native OS file picker dialog for selecting input images.
threading	Enables non-blocking G-code transmission so the UI stays responsive while drawing is in progress.

Drawing Modes

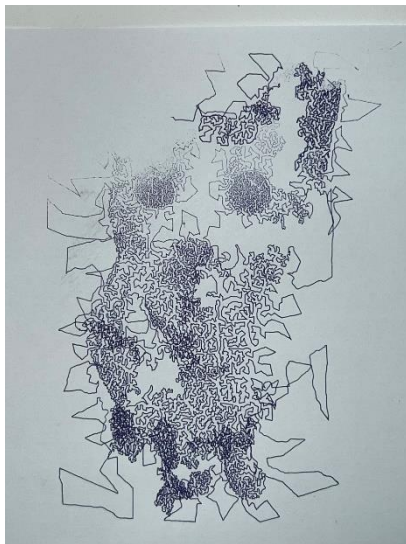
- Squiggle / Sin Wave — Converts image brightness into oscillating sine wave lines filling the drawing area depending on the pixel's brightness and following the sin equation.



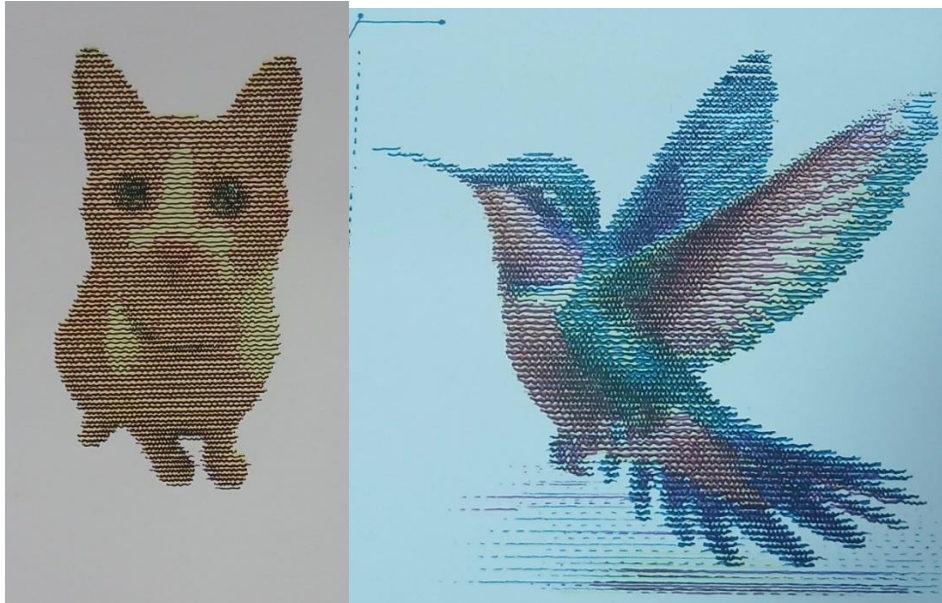
- Vector / Canny Edge — Detects image edges via OpenCV's Canny algorithm and draws the extracted contours.



- Voronoi TSP (Travelling Salesman Problem) — Samples key points and connects them in an optimized continuous path using Pillow-based TSP approximation after the sample points is generated through Voronoi Stippling to create a stippled dot-art effect using Pillow and other modules (removed because of the huge time complexity over the large n points).



- **CMYK Layers** — Separates the image into four color channels and produces one drawing pass per channel for multi-color output.



Buffered Streaming Protocol

Instead of sending G-code line by line and waiting for an OK response after each command, the application continuously fills the GRBL serial receive buffer with upcoming instructions. This keeps the plotter moving without pause, eliminating the jitter caused by line-by-line transmission.

Controllable G-code Streaming

The streaming pipeline is fully controllable at runtime, giving the user direct authority over the drawing process at any point. A dedicated threading mechanism runs the transmission loop in the background, keeping the interface responsive while G-code is being sent. When the user triggers a pause, the pipeline stops feeding new commands into the buffer and sends GRBL's real-time feed-hold command ('!'), which brings the machine to a graceful stop without losing its position. Resuming sends the cycle-start command ('~'), and the buffer refills seamlessly from where it left off — no lines are skipped and no coordinates are lost. This makes it practical to pause mid-drawing to change a pen, inspect the paper, or correct an issue, then continue the same drawing without starting over.

Simple User Interface



The application features a lightweight graphical interface built with Pygame, designed to keep the user's interaction simple and focused. The user can select a drawing style and adjust few style based parameters— such as line density, or color separation settings — without needing to interact with the underlying pipeline directly. The interface acts as a clean entry point to the full processing and transmission workflow, abstracting away the technical complexity while still giving the user meaningful control over the final output.

6. Challenges & Solutions

Challenge	Problem	Solution
Firmware Compatibility	Official GRBL does not support CoreXY kinematics and servo control simultaneously.	Tested multiple community-modified GRBL builds until finding one that supports both CoreXY and PWM servo output.
Overheating Drivers	A4988 drivers overheated, causing missed steps and erratic movement.	Replaced with TMC2208 and added active cooling — runs significantly cooler and quieter.
Pen Pressure	Excess weight on the pen holder caused too much force on the paper, leading to motor stalls.	Designed a bearing-based suspension arm that absorbs excess force rather than transferring it to the paper.
Insufficient Power	12V / 3A supply caused torque loss and unstable motor behavior under load.	Upgraded to 24V / 5A after calculating actual draw (~1.2A per motor).
G-code Lag	Sending commands one-by-one with OK polling caused choppy, interrupted plotter motion.	Implemented buffered streaming: the pipeline pre-fills GRBL's receive buffer for continuous motion.
Color-Change Motor Force	The NEMA 17 motor on the Z-axis cannot generate enough torque to overcome the spring resistance in the fan-style pen-selector mechanism, preventing pens from being pressed down fully.	Two options under evaluation: (1) upgrade to a higher-torque motor, or (2) replace the spring with a weaker one that the NEMA 17 can compress reliably.

7. Results

The completed CNC plotter operates stably with the H-Bot CoreXY structure. Key outcomes include:

- Consistent and accurate drawing across multiple test images and styles.
- Multi-color CMYK drawing with separate passes per channel.
- Smooth, continuous motion with no mid-path pauses thanks to buffered streaming.
- A functional Pygame-based GUI for quick style selection and plotter control.
- Reliable pen pressure management with no paper damage across extended runs.



8. Future Improvements

- Complete the color-change mechanism (in progress) — finalize motor or spring selection to enable fully autonomous multi-color drawing.
- Improved TSP and Voronoi algorithms for higher visual fidelity on complex images.
- Web-based interface to replace the Pygame GUI for easier cross-platform access.
- Live path preview showing the estimated drawing route before sending to the plotter.
- Closed-loop feedback using encoders for improved positioning accuracy.
- Support for additional pen types with configurable pressure profiles.

CNC Plotter

Worked By: Hamza Abo Alzahab , Abdalrhman Rasheed

Coach : Tareq AlSabbagh Altajy

Huge Thanks for the Club Support and for Abdalrhman Kassuma.